TRAZABILIDAD SPRINT 5 Y SPRINT 6

ÓrbiKa — cancelación controlada, confianza y perfil

Brant Antony Chata Choque
· Renzo Fernando Loyola Vilca

AGENDA

Qué vamos a exponer

1. Sprint 5 — alcance y RF-17 Cancelación controlada
2. La transacción atómica: cancelar_pedido_transaccion
3. Matriz de pruebas y DoD del Sprint 5
4. Sprint 6 — alcance, RF-09 y RF-14
5. Brechas encontradas: promedio del vendedor y perfil
6. Matriz de pruebas y DoD del Sprint 6
7. Hallazgos, acciones y conclusiones

Sprint 5 — Cancelación controlada

31 de octubre – 2 de noviembre de 2026 · 3 puntos de historia · RF-17

Objetivo: cancelar un pedido elegible y devolver las unidades al inventario sin duplicar stock.

Compromiso del Sprint 5

Campo	Definición
Requisito comprometido	RF-17 — Cancelación controlada (CU-17)
Dependencia	Pedido creado en el Sprint 4 y sus estados
Entregable	Cancelación con estado trazable y reposición de inventario comprobada
Criterio de cumplimiento	Un pedido cancelable pasa a cancelado y repone la cantidad una sola vez; un pedido entregado no puede cancelarse
Evidencia planificada	Estado del pedido y del stock antes y después, con prueba de un segundo intento
Renzo / Brant	Renzo: reglas de estado y reposición dentro de una operación segura. Brant: validar cancelación antes y después de la entrega, y su repetición

De la interfaz a la base de datos

Frontend — OrderActions.jsx

accionesPermitidas(rol, estado) muestra “Cancelar pedido” solo al comprador y solo en creado o preparando. Un modal accesible exige el motivo y envía PATCH /pedidos/:id/cancelar.

Backend — ruta y servicio

PATCH /api/pedidos/:id/cancelar con requireRole(“comprador”). cancelarPedido() valida el id y el motivo (obligatorio, máx. 300) y delega a db.rpc('cancelar_pedido_transaccion').

Base de datos — migración 003

Función SQL transaccional: bloqueo de fila, pertenencia, estado cancelable, motivo, cambio a cancelado, reposición de stock y aviso al vendedor.

Coincidencia con el SRS

El CU-17 del SRS (9 pasos y 7 alternativas) está entre los mejor detallados del documento, y el código sigue su narrativa en cada rama: pedido no cancelable, cancelación repetida, motivo vacío y error durante la operación. Recuperar stock no vuelve a publicar un producto retirado ni habilita uno ya vencido.

cancelar_pedido_transaccion paso a paso

1. Bloquea la fila del pedido (SELECT ... FOR UPDATE) para que dos cancelaciones simultáneas no se pisen.

2. Verifica que el pedido pertenezca al comprador (“no te pertenece”) y que no esté ya cancelado: si lo está, devuelve el resultado sin repetir la operación (idempotencia).

3. Rechaza los estados no cancelables (“ya no puede cancelarse”) y exige un motivo no vacío.

4. Actualiza el pedido a cancelado con motivo y fecha, repone productos.stock += cantidad y notifica al vendedor — todo en una sola transacción.

8 de 8 pruebas de cancelación superadas

8/8
Pruebas de cancelación en backend

91/91
Backend completo (Vitest)

49/49
Frontend completo (Vitest)

10/12
Casos CP-S05 con PASS real

Prueba real	Qué verifica
database.test.js (PGlite, PostgreSQL real)	Cancelar dos veces repone el stock y avisa una sola vez; no reactiva una publicación retirada ni un producto vencido; no cancela un pedido listo para entrega; un comprador ajeno no puede cancelar
pedido.service / validation / security-migration	El servicio delega a la función transaccional; el motivo es obligatorio; cancelación y reposición ocurren en la misma función
order.helpers.test.js (frontend)	El botón de cancelar solo se habilita en los estados correctos (10 casos)

Casos de prueba CP-S05

ID	Prueba y resultado esperado	Resultado
CP-S05-01	Cancelar pedido propio creado/preparando → cancelado y stock repuesto una vez	PASS — real
CP-S05-02	Segunda cancelación del mismo pedido: no repone stock ni duplica el aviso	PASS — real
CP-S05-03	Cancelar no reactiva una publicación retirada por el vendedor	PASS — real
CP-S05-04	Cancelar un pedido de un producto vencido no lo vuelve disponible	PASS — real
CP-S05-05	No se puede cancelar un pedido listo para entrega o posterior	PASS — real
CP-S05-06	Un comprador no puede cancelar el pedido de otro	PASS — real
CP-S05-07	La cancelación sin motivo es rechazada	PASS — real
CP-S05-08	El servicio delega a la función transaccional de base de datos	PASS — real
CP-S05-09	Cancelación y reposición ocurren en una sola función transaccional	PASS — real
CP-S05-10	La interfaz ofrece “Cancelar pedido” solo en creado y preparando	PASS — real
CP-S05-11	La ruta /cancelar solo acepta al rol comprador (integración HTTP)	Pendiente
CP-S05-12	OrderActions.jsx abre el modal y envía la cancelación (prueba de interfaz)	Pendiente

Cierre del Sprint 5

Criterio DoD	Estado
Criterios de aceptación	Mayormente — falta la prueba HTTP del rol comprador y la prueba del componente de interfaz
Pruebas y resultado registrados	Sí — 140 pruebas reales (91 backend + 49 frontend), 8 específicas de cancelación
Cambios integrados	Sí — ruta, controlador, servicio, validación y función SQL en la rama principal
Sin defectos críticos abiertos	No verificable — el repositorio no expone registro de incidencias
Evidencia sin datos sensibles	Sí

El objetivo central — cancelación con reposición de stock trazable y sin duplicados — se cumple en el código. Para cerrar el DoD faltan dos pruebas por agregar (HTTP e interfaz), no cambios de funcionalidad.

Hallazgos y acciones del Sprint 5

N.°	Hallazgo	Acción recomendada	Resp.
H-01	Sin prueba de integración HTTP para PATCH /pedidos/:id/cancelar	Agregar caso a app.test.js: vendedor recibe 403, comprador dueño recibe 200	Brant
H-02	Sin prueba de componente para OrderActions.jsx; solo la función pura accionesPermitidas()	Prueba con Testing Library: el botón abre el modal y confirmar llama al endpoint	Renzo
H-03	Sin .env, 3 de 10 suites del backend no cargan	Documentar variables de prueba o versionar un .env.test de ejemplo	Renzo
H-04	Un único commit contiene el código de varios sprints	Commits incrementales por sprint para trazabilidad cronológica	Ambos

Sprint 6 — Confianza y perfil

2–7 de noviembre de 2026 · 6 puntos de historia · RF-09, RF-14

Objetivo: reputación basada en pedidos completados y datos del titular actualizados.

Compromiso del Sprint 6

Campo	Definición
Requisitos comprometidos	RF-09 (calificar transacción, CU-06) y RF-14 (editar perfil de usuario, CU-14)
Dependencia	Pedidos e identidad validados en los Sprints 1 y 4
Entregable	Perfil editable y registro de calificación ligado a una transacción completada
Criterio de cumplimiento	Solo el titular modifica su perfil; solo el comprador del pedido completado califica una vez, con puntaje de 1 a 5
Evidencia planificada	Capturas de perfil actualizado y de registro de calificación; rechazos cubiertos en el reporte de prueba
Renzo / Brant	Renzo: edición de perfil con control de titularidad y calificación 1–5. Brant: probar perfil ajeno, pedido pendiente, calificación repetida y puntajes fuera de rango

RF-09: el registro existe, el promedio no

Backend — calificarPedido()

Exige rol comprador, valida el puntaje (entero de 1 a 5), comprueba que el pedido sea del comprador y esté completado, e inserta en calificaciones. Ruta: POST /api/pedidos/:id/calificacion.

Base de datos — calificaciones

pedido_id es UNIQUE: la base impide más de una calificación por pedido aunque la validación de la aplicación fallara. Una inserción rechazada se informa como “ya fue calificado”.

Brecha frente al SRS

El paso 8 de CU-06 exige recalcular el promedio del vendedor. No existe vista, consulta ni campo que lo calcule: producto.service.js solo trae nombre_negocio y ubicación.

Cobertura de pruebas reales

Solo la validación aislada del puntaje está probada (“limita puntaje y comentario”: acepta 1–5, rechaza 6). calificarPedido() no tiene ninguna prueba: ni rechazo por pedido ajeno, ni por pedido no completado, ni por calificación duplicada, aunque esas ramas sí están programadas.

RF-14: no está implementado

Dónde se buscó	Qué se encontró
Backend — backend/src/app.js	Solo monta /api/imagenes, /api/productos y /api/pedidos. No existe router, controlador ni servicio de perfiles
Frontend — páginas	Ninguna página “Mi perfil” entre las 12 páginas (Auth, Home, Login, MyOrders, ProductDetail, ProductList, etc.)
Frontend — AuthContext.jsx	Solo lee el perfil al iniciar sesión y permite cambiar la contraseña; ninguna función escribe en la tabla perfiles
Base de datos	Existe la política RLS “Perfiles: cada quien ve y edita el suyo”, que permitiría la edición, pero ningún código del cliente la invoca
Pruebas	Ninguna: no hay código que una prueba pueda ejercitar

Es una diferencia real entre el Sprint Backlog (“Renzo: habilitar edición de perfil…”) y el repositorio; conviene resolverla o comunicarla antes de la entrega.

Casos de prueba CP-S06

ID	Req.	Prueba y resultado esperado	Resultado
CP-S06-01	RF-09	El puntaje debe ser entero de 1 a 5; fuera de rango es rechazado	PASS — real
CP-S06-02	RF-09	Calificar un pedido propio completado registra puntaje y comentario	Pendiente
CP-S06-03	RF-09	No se puede calificar el pedido de otro comprador	Pendiente
CP-S06-04	RF-09	No se puede calificar un pedido aún no completado	Pendiente
CP-S06-05	RF-09	No se puede calificar dos veces el mismo pedido (UNIQUE en base)	Pendiente
CP-S06-06	RF-09	El promedio del vendedor se recalcula y se muestra tras calificar	Pendiente
CP-S06-07	RF-14	Editar nombre, teléfono, foto y dirección desde “Mi perfil”	Sin código
CP-S06-08	RF-14	Solo el titular puede modificar su propio perfil	Sin código
CP-S06-09	RF-14	El vendedor edita nombre y ubicación de su negocio	Sin código
CP-S06-10	RF-14	Correo duplicado o nombre vacío son rechazados al guardar	Sin código

1 caso con PASS real · 5 pendientes por falta de pruebas sobre código existente (RF-09) · 4 sin código que probar (RF-14).

Cierre del Sprint 6

Criterio DoD	Estado
Criterios de aceptación	No — RF-09 sin promedio del vendedor ni pruebas del servicio; RF-14 sin código
Pruebas y resultado registrados	Parcialmente — 140 pruebas reales en total, solo 1 de RF-09 y ninguna de RF-14
Cambios integrados	Parcialmente — calificación integrada; edición de perfil no existe
Sin defectos críticos abiertos	No verificable — sin registro de incidencias
Evidencia sin datos sensibles	Sí

A diferencia de los Sprints 4 y 5, aquí hay una brecha real entre lo planificado y lo construido. Las 140 pruebas que pasan son las mismas del commit único: no se agregó ninguna prueba nueva para este sprint.

Hallazgos y acciones del Sprint 6

N.°	Hallazgo	Acción recomendada	Resp.
H-01	RF-14 no tiene ruta, página ni servicio; solo la política RLS	Implementar PATCH /api/perfiles y la página “Mi perfil” según CU-14	Renzo
H-02	El promedio del vendedor (paso 8 de CU-06) no se calcula en ningún lugar	Vista o consulta de promedio por vendedor y exponerla en producto.service.js	Brant
H-03	calificarPedido() no tiene pruebas reales	Pruebas con mock y PGlite: válida, ajeno, no completado y repetida	Ambos
H-04	Sin .env, 3 de 10 suites del backend no cargan	Documentar variables de prueba o versionar un .env.test	Renzo
H-05	Backlog y repositorio no coinciden: un solo commit y RF-14 sin construir	Commits incrementales por sprint para detectar brechas a tiempo	Ambos

CONCLUSIONES

Sprint 5 sólido, Sprint 6 con brechas

Sprint 5 — RF-17

Cancelación atómica, idempotente y protegida contra reactivaciones indebidas. 10 de 12 casos con PASS real; los 2 pendientes son niveles de prueba (HTTP e interfaz), no funcionalidad faltante.

Sprint 6 — RF-09 y RF-14

RF-09 registra la calificación pero no calcula el promedio del vendedor y casi no tiene pruebas. RF-14 no está implementado. Solo 1 de 10 casos con PASS real.

Próximos pasos antes de la entrega

1) Implementar RF-14 (ruta PATCH de perfiles y página “Mi perfil”). 2) Calcular y mostrar el promedio del vendedor. 3) Agregar pruebas de calificarPedido(), de la ruta /cancelar y de OrderActions.jsx. 4) Mantener commits incrementales por sprint.

Gracias

Preguntas

Grupo G01 · Construcción de Software I · Universidad Privada de Tacna